好的，这是根据您提供的文档内容，对86个问题进行的简要要点回答。回答力求简洁、准确，并直接对应问题核心。

第一章：基本概念与架构

1. **分布式系统的定义及特点?**
   - **定义** ： 若干独立计算机的集合，对用户表现为单一系统。
   - **特点** ： 物理分布、逻辑统一；资源共享；透明性；开放性；可扩展性；容错性。
2. **分布式系统透明性主要包括哪几方面?**
   - 访问透明性、位置透明性、迁移透明性、重定位透明性、复制透明性、并发透明性、故障透明性、持久化透明性。（共8种）
3. **策略和机制有什么不同?**
   - **机制** ： *如何做* ，是系统提供的具体实现方法（如时钟同步算法）。
   - **策略** ： *做什么* ，是使用机制的具体规则或决策（如“若时钟偏差大于阈值则调整”）。
4. **分布式系统的可扩展性包括哪些方面?有哪些技术可以实现可扩展性?**
   - **方面** ： 规模可扩展、地理可扩展、管理可扩展。
   - **技术** ： 复制与缓存、内容分发网络（CDN）、异步通信、分片（Sharding）、P2P架构。
5. **集群系统和网格之间的区别?**
   - **集群** ： 同构、集中管理、高速网络、逻辑上紧耦合（如超级计算机）。
   - **网格** ： 异构、分散管理、广域网、逻辑上松耦合（如跨机构的计算资源共享）。
6. **云计算作为一种新的计算模式适用于所有企业?**
   - **不适用所有**。 适用于追求弹性、低成本、免运维的中小企业或特定场景。不适合数据敏感性极高、有严格合规要求或已有大量遗留系统的企业。
7. **主要的软件体系结构包括哪几种?有什么特点?**
   - **分层架构** ： 层次分明，易于管理，但性能可能成瓶颈。
   - **面向对象架构** ： 组件化，易于复用和扩展。
   - **基于事件架构** ： 解耦性好，适合异步通信。
   - **数据共享架构** ： 以数据为中心，如黑板系统。
8. **主要的系统体系结构包括哪几种?有什么特点?与软件体系结构的主要区别是什么?**
   - **集中式架构** ： C/S模式，简单但单点故障。
   - **非集中式架构** ： P2P模式，去中心化，健壮但复杂。
   - **混合架构** ： 结合两者优点，如边缘计算。
   - **与软件体系结构区别** ： 系统体系结构关注物理节点部署和网络连接，软件体系结构关注代码模块的组织和交互。
9. **分布式的分层结构**
   - 典型分层： 硬件/网络层 -> 操作系统层 -> 中间件层 -> 应用程序层。中间件层是实现分布式透明性的关键。
10. **非集中化的体系结构主要类型包括哪些?**
    - 结构化P2P（如Chord, Pastry）、非结构化P2P（如Gnutella）、层次化P2P。

第二章：进程、通信与命名

1. **Chord结构的生成和查找算法?**
    - **生成** ： 节点通过哈希（如SHA-1）被映射到环形标识符空间。
    - **查找** ： 通过维护指向环上后继节点的指针（Finger Table）进行高效路由，查询复杂度为O(log N)。
2. **非结构化的点对点系统搜索内容的方式?**
    - **泛洪** ： 向所有邻居转发查询，直到找到内容或TTL耗尽。
    - **随机漫步** ： 随机选择邻居转发查询。
    - **索引** ： 某些节点维护内容索引。
3. **什么是超级对等节点?如何确定超级节点?**
    - **超级节点** ： 网络中能力较强（带宽、计算、在线时间）的普通节点，承担索引、路由等更多责任。
    - **确定方式** ： 基于节点的性能指标（如带宽、稳定性）动态选举或自行宣告。
4. **在P2P系统中节点之间连接的方式?**
    - TCP或UTP协议。结构化P2P遵循特定拓扑（如环、树），非结构化P2P是随机图。
5. **分布式系统的自我管理?**
    - 系统自动处理配置、修复、优化、保护，无需人工干预。目标包括自配置、自修复、自优化、自保护。
6. **分布式系统中为什么利用线程而不是进程?**
    - 线程创建和切换开销远小于进程；线程间共享内存，通信效率高；更易实现并发服务器。
7. **虚拟机化主要包含哪些方式，简要描述?**
    - **进程虚拟机** ： 如JVM，在OS上模拟指令集。
    - **原生虚拟机监控器** ： 如Xen，直接在硬件上运行，支持多种Guest OS。
    - **主机虚拟机监控器** ： 如VMware Workstation，作为Host OS上的应用程序运行。
8. **在客户端-服务器模型中，服务器的状态主要分为几种，简要解释。**
    - **无状态服务器** ： 不保存任何客户端状态信息，每个请求独立（如HTTP RESTful API）。
    - **有状态服务器** ： 在请求间维护客户端会话状态（如FTP）。
9. **简述如何实现代码迁移?代码迁移的模型?如何区分强迁移和弱迁移?**
    - **实现** ： 将代码及其执行状态（弱迁移则不包含）打包并传输到目标机器。
    - **模型** ： 客户端-服务器、代码按需、远程评估、移动代理。
    - **强/弱迁移** ： **强迁移** 迁移代码和完整的执行状态；**弱迁移** 只迁移代码。
10. **虚拟机迁移的种类及主要特点?**
    - **冷迁移** ： 虚拟机关机后迁移。简单，但服务中断。
    - **热/活迁移** ： 虚拟机运行时迁移。服务不中断，但技术复杂。
11. **RPC远程过程调用的概念及主要步骤?**
    - **概念** ： 使调用远程过程像调用本地过程一样透明的通信机制。
    - **步骤** ： 1. 客户端存根打包参数；2. 传输至服务器；3. 服务器存根解包并调用真实函数；4. 将结果按相反路径返回。
12. **使用socket进行网络通信的主要模式及主要过程**
    - **模式** ： 面向连接的TCP和无连接的UDP。

- **过程** ： 服务器：socket() -> bind() -> listen() -> accept() -> recv()/send() -> close()。 客户端：socket() -> connect() -> send()/recv() -> close()。

13. **什么是点对点通信、广播和多播?**

    - **点对点** ： 一对一通信。
    - **广播** ： 一对所有通信。
    - **多播** ： 一对一组（特定集合）通信。

14. **简述Gossip数据通信和反熵通信模型?**

    - **Gossip** ： 节点随机选择其他节点交换信息，以概率最终达成一致。
    - **反熵** ： 通过比较和同步数据副本的差异来消除不一致性。

15. **Chord指纹表的查找过程，节点如何加入和退出指纹表?**

    - **查找** ： 查询节点在Finger Table中找到最大但不大于目标ID的后继节点，将查询转发给它，重复直至找到。
    - **加入** ： 新节点通过已知节点加入环，初始化自己的Finger Table，并通知相关节点更新它们的表。
    - **退出** ： 正常退出前通知后继和前驱节点更新链接。异常退出通过后继节点的周期性检测来处理。

第三章：命名、同步与一致性

1. **简述在树结构目录中查询实体及插入实体的过程?**

    - **查询** ： 从根目录开始，沿路径逐层解析目录名，直到找到目标文件。
    - **插入** ： 沿路径找到父目录，在其中创建新的目录项。

2. **名称解析闭包?**

    - 名称解析过程中涉及的名称服务器的集合。

3. **实体的硬链接和软链接?**

    - **硬链接** ： 多个目录项直接指向同一个i-node（数据块）。删除一个链接不影响数据。
    - **软链接/符号链接** ： 一个特殊的文件，内容为另一个文件的路径。源文件删除后，软链接失效。

4. **什么是挂接点和挂载点?**

    - **挂接点** ： 一个目录，用于将另一个文件系统（如U盘）连接到当前文件系统树中。
    - **挂载点** ： 挂接点所在的目录。

5. **什么是命名空间,命名空间的分层结构主要由哪几部分构成?**

    - **命名空间** ： 所有名称的集合。
    - **分层结构** ： 全局层、行政层、管理域层。

6. **迭代命名解析和递归命名解析?**

    - **迭代解析** ： 本地名称服务器依次查询各级服务器，最终将结果返回客户端。
    - **递归解析** ： 本地名称服务器代表客户端完成所有查询，最终将最终答案返回客户端。

7. **瞬时同步通信的主要问题以及解决方案?**

    - **问题** ： 发送方可能阻塞等待接收方，导致性能差和易死锁。
    - **方案** ： 使用异步通信（如消息队列）、缓冲、超时机制。

8. **时钟同步:内同步和外同步;如何计算时钟偏差?**

    - **内同步** ： 进程间时钟差值有界。
    - **外同步** ： 进程时钟与UTC时间差值有界。

- **计算偏差** ： 通过交换消息的时间戳，利用公式 `θ = [(T2 - T1) + (T3 - T4)] / 2`和 `δ = (T4 - T1) - (T3 - T2)`计算偏差和往返延迟。

9. **如何在没有UTC的情况下保障时间的准确性?**
   - 使用内部同步算法，如Berkeley算法，选择一个主节点，收集其他节点时间，计算平均值并通知各节点调整。
10. **Happen-before关系**
    - 定义事件间的偏序关系。如果事件a在事件b之前发生，则a -> b。用于确定事件的因果关系。
11. **逻辑时钟?及算法**
    - **概念** ： 为事件分配单调递增的编号，用于捕获Happen-before关系。
    - **Lamport算法** ： 1. 本地事件，时钟+1；2. 发送消息时，带上当前时钟值；3. 接收消息时，时钟 = max(本地时钟, 消息时间戳) + 1。
12. **向量时钟?及算法**
    - **概念** ： 为每个进程维护一个向量，记录所有进程的逻辑时钟知识，能准确判断因果关系和并发。
    - **算法** ： 每个进程维护向量V。1. 本地事件：Vi[i]++；2. 发送消息：带上整个向量V；3. 接收消息：Vj[k] = max(Vj[k], Vm[k]) for all k，然后Vj[j]++。
13. **什么是全序广播?**
    - 保证所有正确进程以相同的顺序交付所有消息的广播协议。是实现状态机复制的基石。
14. **如何利用Lamport逻辑时钟解决互斥访问及临界区访问?**
    - 进程在请求进入临界区时，向所有进程发送带有时间戳的请求。当收到所有进程的回复（且时间戳均大于自己的请求）时，方可进入临界区。时间戳用于打破平局。
15. **因果有序的多播传播?**
    - 保证有因果关系的消息（如发送和接收）必须按因果顺序被所有进程交付。并发消息可以按不同顺序交付。通常用向量时钟实现。

第四章：容错与高级主题

1. **解决分布式系统中多进程互斥的方法?有什么特点?过程是怎样的?**
   - **集中式算法** ： 简单，单点故障。
   - **分布式算法** ： 需要2(N-1)条消息，延迟高。
   - **令牌环算法** ： 公平，但令牌丢失难以处理。
2. **Ricart& Agrawala互斥算法?**
   - 基于权限的分布式算法。进程想进入临界区时，向所有其他进程发送带时间戳的请求。仅当收到所有进程的同意回复后，才能进入。收到请求的进程，如果自己不想进入或对方时间戳更小，则立即回复同意，否则推迟回复。
3. **典型的选举算法有哪些?**
   - Bully算法（基于ID大小）、Ring算法（基于令牌环）。
4. **无线网络中的选举算法;**
   - 基于能量、信号强度等指标，而非节点ID。
5. **数据一致性模型**

- 数据存储对进程的承诺，规定读操作可以返回什么值。

6. **连续一致性或者一致性程度主要包含哪些方面?**
   - 数值偏差、新旧偏差、更新顺序偏差。
7. **什么是顺序一致性和因果一致性?**
   - **顺序一致性** ： 所有进程看到的操作序列一致，且每个进程自身的操作顺序得到保留。
   - **因果一致性** ： 只保证有因果关系的操作顺序一致。
8. **数据为中心的一致性和客户为中心的一致性模型各自适用于什么场景?**
   - **数据为中心** ： 适用于对数据一致性要求高的读写并重系统，如数据库。
   - **客户为中心** ： 适用于读为主、最终一致性可接受的系统，如DNS、Web缓存。
9. **什么是最终一致性?有什么优缺点?**
   - **定义** ： 如果不再有更新，最终所有副本会变得一致。
   - **优点** ： 可用性高、延迟低。
   - **缺点** ： 存在不一致时间窗口。
10. **读写一致性?写读一致性?具体场景?**
    - **读写一致性** ： 用户总能读到自己的最新写入。场景：发帖后立刻查看。
    - **写读一致性** ： 用户读到的数据不会旧于之前读到的数据。场景：翻页浏览。
11. **服务器副本的放置位置?**
    - 永久副本（静态放置）、服务器发起的副本（动态调整）、客户端发起的副本（缓存）。
12. **内容分发的方式有哪些?Pull-based和Push-based的内容分发方法有什么不同?**
    - **Pull-based** ： 客户端主动拉取。适合读少写多，一致性可能滞后。
    - **Push-based** ： 服务器主动推送。适合读多写少，一致性高，但服务器负载大。
13. **在一致性协议中，如何限定数值偏差?限制新旧偏差?**
    - 通过“连续一致性”模型，使用偏差阈值（如δ）来量化允许的不一致程度。协议通过 gossip 机制传播更新，确保偏差不超过阈值。
14. **基于主备协议的复制协议主要包括哪两种方式?分别有什么特点?**
    - **远程写协议** ： 所有写操作经过主副本，同步到备份。强一致性，但写延迟高。
    - **本地写协议** ： 写操作可迁移到本地副本，该副本成为新的主副本。灵活性高，但复杂性高。
15. **如何理解基于团体的复制写协议?**
    - 即Quorum协议。设置读法定数Nr和写法定数Nw，满足Nr + Nw > N（总副本数）且Nw > N/2，通过多数表决来保证读写操作能感知到彼此，从而实现一致性。
16. **什么是可依赖性?与可依赖性相关的需求包括哪些方面?**
    - **可依赖性** ： 系统提供可信赖服务的能力。
    - **相关需求** ： 可用性、可靠性、安全性、可维护性。
17. **可靠性与可用性的定义及区别?**
    - **可靠性** ： 系统在给定时间内无故障连续运行的能力（MTTF衡量）。
    - **可用性** ： 系统准备就绪可供使用的比例（A = MTTF / ((MTTF + MTTR) ）。
    - **区别** ： 可靠性关注“不停机”，可用性关注“能用”。

18. **分布式系统中主要包含哪些失效模型?简要描述。**

    - 崩溃失效、遗漏失效、定时失效、响应失效、任意/拜占庭失效。
    - 分布式系统中冗余的主要方式有哪些？

    在分布式系统中如何检测失效？ 在分布式系统中，主要通过**超时心跳机制**来探测失效，即若监控方在预定时间内未收到目标节点的存活信号，则判定其失效；为提升准确性，常结合 **分布式共识** （如Gossip协议或仲裁投票）来避免误判，但该方法本质上无法完全区分节点崩溃与网络分区。

19. **分布式系统中冗余的主要方式有哪些?**

    - 信息冗余、时间冗余、物理冗余（硬件/软件副本）。

20. **如何设计可靠的RPC通信机制?**

    - 幂等性操作、超时与重传、去重机制、使用至少一次/至多一次语义。

21. **什么是可靠多播?可靠多播存在的问题?**

    - **定义** ： 保证消息被所有非故障进程接收。
    - **问题** ： Ack/NAK内爆、可扩展性差。

22. **简述两阶段提交协议?在参与者失效时如何恢复?协作者失效时如何恢复?**

    - **阶段1** ： 协调者询问所有参与者是否能提交。
    - **阶段2** ： 如果全部同意，协调者发送提交命令；否则发送中止命令。
    - **参与者失效** ： 协调者超时后中止事务。
    - **协调者失效** ： 参与者阻塞，等待新协调者选举后恢复状态。

23. **分布式系统失效恢复的主要方式?**

    - **回滚恢复** ： 使用检查点和消息日志回滚到一致状态。
    - **前向恢复** ： 通过冗余信息在出错状态下继续前进。

24. **Client-server失效场景及恢复方法?**

    - **场景** ： 客户端/服务器崩溃、网络分区。
    - **方法** ： 幂等操作、会话状态持久化、租约机制、超时检测。

25. **什么是恢复线?**

    - 最近一次全局一致的检查点集合，是系统回滚的目标。

26. **什么是检查点方法?**

    - 定期将进程状态保存到稳定存储。

27. **独立检查点方法?协调检查点方法?**

    - **独立检查点** ： 各进程独立打检查点，可能产生多米诺效应。
    - **协调检查点** ： 全局协调所有进程同时打检查点，保证全局一致性，但可能阻塞。

28. **什么是共识?主要的共识协议有哪些?共识的使用场景有哪些?**

    - **共识** ： 所有非故障进程就某个值达成一致。
    - **协议** ： Paxos, Raft（用于非恶意故障）；PBFT（用于拜占庭故障）。
    - **场景** ： 领导者选举、分布式锁、状态机复制、区块链。

29. **Paxos的主要过程? Paxos算法的特性? Paxos的主要变种?**

    - **过程** ： 准备阶段、接受阶段。

- **特性** ： 安全性高，但难以理解，活性可能有问题。
- **变种** ： Multi-Paxos, Cheap Paxos, Fast Paxos。

30. **Raft共识的主要特点是什么?Raft的主要过程。Raft和Paxos的主要区别有哪些?**

- **特点** ： 易于理解，强领导者模型。
- **过程** ： 领导者选举、日志复制。
- **区别** ： Raft将共识问题分解为更清晰的子问题（选举、日志、安全），而Paxos更抽象。

31. **Paxos和 Raft适用的故障场景是什么?**

- 崩溃失效（Crash Fault），非拜占庭失效。

32. **哪些共识协议可以解决拜占庭失效的问题?**

- 实用拜占庭容错算法，如PBFT。

33. **NFS文件系统的主要架构?**

- 无状态服务器、客户端缓存、基于RPC的通信。

34. **GFS文件系统的主要特点?**

- 单Master（元数据管理）、大文件分块（Chunk）、多副本、为MapReduce优化。

35. **什么是文件系统语义模型?主要的语义模型包括哪些?简要描述其特征。**

- **Unix语义** ： 读操作看到最近一次写的结果。
- **会话语义** ： 文件关闭后，修改才对其他客户端可见。
- **事务语义** ： 一系列操作原子执行。

36. **拜占庭容错的基本思想及基本过程?**

- **思想** ： 在存在恶意节点的情况下，通过多数投票（3f+1个节点容忍f个恶意节点）达成共识。
- **过程** ： 三阶段协议：预准备、准备、提交。

37. **P2P系统中用于提高系统可用性的方案?以及方案的主要特点。**

- **方案** ： 动态复制、缓存、路径冗余、超级节点。
- **特点** ： 去中心化、自组织、高冗余。

38. **在分布式文件系统中主要关注的问题包括哪些?并分别给出一些解决问题的方案。**

- **元数据管理** ： 集中式（如GFS）、分布式（如Ceph）。
- **一致性** ： 使用副本协议（如Quorum）。
- **容错** ： 副本、擦除码。

39. **分布式机器学习的参数服务器如何理解?**

- 一种编程框架。工作节点（Worker）计算梯度，参数服务器（Server）节点集中存储和更新模型参数，实现异步并行训练。

40. **如何保障参数服务器状态的一致性?**

- **同步更新** ： 所有Worker完成一次计算后同步更新参数。一致性强，但慢。
- **异步更新** ： Worker计算完立即更新。速度快，但存在过期梯度问题，最终一致。

41. **MapReduce计算泛型的主要流程是什么?**

- 1. Input Splitting -> 2. Mapping -> 3. Shuffling & Sorting -> 4. Reducing -> 5. Final Result。

42. **MapReduce计算泛型的主要特点是什么?**

- 自动并行化、容错、数据局部性、简洁的编程接口。

43. **与传统的高性能计算相比，MapReduce的优势是什么?**

    - 对程序员更友好，自动处理故障、数据分布和并行调度，适合海量数据批处理。

44. **Map和Reduce的原理是什么?**

    - **Map** ： 将输入键值对映射为一组中间键值对。`(k1, v1) -> list(k2, v2)`
    - **Reduce** ： 将相同中间键k2对应的值列表合并。`(k2, list(v2)) -> list(v3)`

45. **K8S的核心组件包括哪些?主要的功能是什么?**

    - **Master节点** ：
    - **API Server** ： 集群管理的唯一入口。
    - **Scheduler** ： 负责Pod的调度。
    - **Controller Manager** ： 维护集群状态（如副本数）。
    - **etcd** ： 保存集群所有网络配置和对象状态的分布式键值存储。
    - **Node节点** ：
    - **kubelet** ： 负责与管理节点通信的代理。
    - **kube-proxy** ： 维护节点上的网络规则。
    - **Container Runtime** ： 负责运行容器（如Docker）。